МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №4

Тема: **«Нефункциональное тестирование»**

Дисциплина: **«Тестирование и отладка программного обеспечения»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: для разработанного приложения (пп. 1.4) продемонстрировать на выбор применение технологии нефункционального тестирования: «Тестирование производительности».

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

1. Анализ требований

Игра представляет собой 2D-аркаду на Pygame с одиночным и мультиплеерным режимами. Ключевые требования к производительности:

1) Целевая частота кадров: 60 FPS (≈16.6 мс на кадр)

2) Логика кадра (обновление сущностей, коллизии, ввод) должна занимать менее 5 мс, чтобы оставался запас для рендеринга

3) Рендеринг должен укладываться в оставшиеся ~11 мс

4) Загрузка уровня не должна вызывать заметных фризов (< 10 мс)

2. Составление тестов

Ниже приведён полный листинг тестов производительности pytest.

```
"""Performance testing for the Coin Collector game using pytest."""
from __future__ import annotations
import statistics
import time
from dataclasses import dataclass, field
from typing import Callable

import pygame
import pytest

from coin_collector.entities import BonusCoin, Coin, Hazard, Player
from coin_collector.factory import ArcadeObjectFactory, ObjectKind
from coin_collector.input_adapter import InputFrame,
PygameInputAdapter
from coin_collector.level import build_default_level,
build_two_player_level
from coin_collector.multiplayer import TwoPlayerMatch
from coin_collector.rendering import PygameDrawingAPI
from coin_collector.settings import SettingsManager


@dataclass
class PerformanceResult:
```

```
    """Results of a performance test."""
    name: str
    iterations: int
    total_time: float
    avg_time: float
    min_time: float
    max_time: float
    std_dev: float
    median: float = 0.0
    p95: float = 0.0
    p99: float = 0.0
    ops_per_second: float = 0.0

    def __str__(self) -> str:
        lines = [
            f"{'=' * 80}",
            f"Test: {self.name}",
            f"{'=' * 80}",
            f"Iterations:         {self.iterations:>10}",
            f"Total Time:         {self.total_time:>10.3f} s",
            f"{'-' * 80}",
            f"Average Time:       {self.avg_time*1000:>10.3f} ms",
            f"Median (P50):       {self.median*1000:>10.3f} ms",
            f"95th Percentile:    {self.p95*1000:>10.3f} ms",
            f"99th Percentile:    {self.p99*1000:>10.3f} ms",
            f"Min Time:           {self.min_time*1000:>10.3f} ms",
            f"Max Time:           {self.max_time*1000:>10.3f} ms",
            f"Std Deviation:      {self.std_dev*1000:>10.3f} ms",
            f"{'-' * 80}",
            f"Throughput:         {self.ops_per_second:>10.0f}
ops/sec",
            f"{'=' * 80}",
        ]
        return "\n".join(lines)


@pytest.fixture(scope="module")
def pygame_env():
    """Initialize pygame environment for all tests."""
    pygame.init()
    screen = pygame.display.set_mode((960, 640))
    drawing = PygameDrawingAPI(screen)
    factory = ArcadeObjectFactory()

    yield {
        "screen": screen,
        "drawing": drawing,
        "factory": factory,
    }

    pygame.quit()


def measure_performance(name: str, func: Callable[[], None],
iterations: int = 100) -> PerformanceResult:
    """Measure the performance of a function."""
    times: list[float] = []
```

```
    # Warm-up
    for _ in range(min(10, iterations)):
        func()

    # Actual measurement
    for _ in range(iterations):
        start = time.perf_counter()
        func()
        end = time.perf_counter()
        times.append(end - start)

    sorted_times = sorted(times)
    total_time = sum(times)
    avg_time = statistics.mean(times)

    # Calculate percentiles
    p50_idx = int(len(sorted_times) * 0.50)
    p95_idx = int(len(sorted_times) * 0.95)
    p99_idx = int(len(sorted_times) * 0.99)

    return PerformanceResult(
        name=name,
        iterations=iterations,
        total_time=total_time,
        avg_time=avg_time,
        min_time=min(times),
        max_time=max(times),
        std_dev=statistics.stdev(times) if len(times) > 1 else 0.0,
        median=sorted_times[p50_idx],
        p95=sorted_times[p95_idx],
        p99=sorted_times[p99_idx],
        ops_per_second=iterations / total_time if total_time > 0 else
0,
    )


class TestEntityCreation:
    """Test performance of entity creation."""

    def test_create_player(self, pygame_env):
        result = measure_performance("Create Player", lambda:
Player((100, 100)), 1000)
        print(result)
        assert result.avg_time < 0.001  # Should be faster than 1ms

    def test_create_coin(self, pygame_env):
        result = measure_performance("Create Coin", lambda: Coin((100,
100)), 1000)
        print(result)
        assert result.avg_time < 0.001

    def test_create_bonus_coin(self, pygame_env):
        def create():
            BonusCoin(Coin((100, 100)))
        result = measure_performance("Create BonusCoin", create, 1000)
        print(result)
        assert result.avg_time < 0.001
```

```
    def test_create_hazard(self, pygame_env):
        result = measure_performance("Create Hazard", lambda:
Hazard((100, 100)), 1000)
        print(result)
        assert result.avg_time < 0.001


class TestEntityUpdates:
    """Test performance of entity updates."""

    def test_update_player(self, pygame_env):
        player = Player((100, 100))
        bounds = pygame.Rect(0, 0, 800, 600)
        frame = InputFrame(movement=(1, 0))

        def update():
            player.update(frame, 0.016, bounds)

        result = measure_performance("Update Player", update, 1000)
        print(result)
        assert result.avg_time < 0.001

    def test_update_coin(self, pygame_env):
        coin = Coin((200, 200))

        def update():
            coin.update(0.016)

        result = measure_performance("Update Coin", update, 1000)
        print(result)
        assert result.avg_time < 0.0001

    def test_draw_hazard(self, pygame_env):
        hazard = Hazard((300, 300))
        drawing = pygame_env["drawing"]

        def draw():
            hazard.draw(drawing)

        result = measure_performance("Draw Hazard", draw, 1000)
        print(result)
        assert result.avg_time < 0.001


class TestFactoryPerformance:
    """Test performance of object factory."""

    def test_factory_create_player(self, pygame_env):
        factory = pygame_env["factory"]

        def create():
            factory.create(ObjectKind.PLAYER, (100, 100))

        result = measure_performance("Factory: Create Player", create,
1000)
        print(result)
        assert result.avg_time < 0.001
```

```
    def test_factory_create_coin(self, pygame_env):
        factory = pygame_env["factory"]

        def create():
            factory.create(ObjectKind.COIN, (100, 100))

        result = measure_performance("Factory: Create Coin", create,
1000)
        print(result)
        assert result.avg_time < 0.001

    def test_factory_create_hazard(self, pygame_env):
        factory = pygame_env["factory"]

        def create():
            factory.create(ObjectKind.HAZARD, (100, 100))

        result = measure_performance("Factory: Create Hazard", create,
1000)
        print(result)
        assert result.avg_time < 0.001


class TestLevelBuilding:
    """Test performance of level building."""

    def test_build_single_player_level(self, pygame_env):
        factory = pygame_env["factory"]

        def build():
            build_default_level(factory, (960, 640))

        result = measure_performance("Build Single Player Level",
build, 100)
        print(result)
        assert result.avg_time < 0.01

    def test_build_two_player_level(self, pygame_env):
        factory = pygame_env["factory"]

        def build():
            build_two_player_level(factory, (960, 640))

        result = measure_performance("Build Two Player Level", build,
100)
        print(result)
        assert result.avg_time < 0.01


class TestRenderingPerformance:
    """Test performance of rendering operations."""

    def test_draw_player(self, pygame_env):
        player = Player((100, 100))
        drawing = pygame_env["drawing"]

        def draw():
            player.draw(drawing)
```

```
        result = measure_performance("Draw Player", draw, 500)
        print(result)
        assert result.avg_time < 0.001

    def test_draw_coin(self, pygame_env):
        coin = Coin((200, 200))
        drawing = pygame_env["drawing"]

        def draw():
            coin.draw(drawing)

        result = measure_performance("Draw Coin", draw, 500)
        print(result)
        assert result.avg_time < 0.001

    def test_draw_multiple_objects(self, pygame_env):
        player = Player((100, 100))
        coin = Coin((200, 200))
        hazard = Hazard((300, 300))
        drawing = pygame_env["drawing"]

        def draw():
            for _ in range(10):
                player.draw(drawing)
                coin.draw(drawing)
                hazard.draw(drawing)

        result = measure_performance("Draw 10 of Each", draw, 100)
        print(result)
        assert result.avg_time < 0.01


class TestCollisionDetection:
    """Test performance of collision detection."""

    def test_check_10_collisions(self, pygame_env):
        player = Player((100, 100))
        coins = [Coin((100 + i * 50, 100)) for i in range(10)]

        def check():
            for coin in coins:
                player.rect.colliderect(coin.rect)

        result = measure_performance("Check 10 Collisions", check,
1000)
        print(result)
        assert result.avg_time < 0.001

    def test_check_100_collisions(self, pygame_env):
        player = Player((100, 100))

        def check():
            many_coins = [Coin((i * 30, i * 30)) for i in range(100)]
            for coin in many_coins:
                player.rect.colliderect(coin.rect)

        result = measure_performance("Check 100 Collisions", check,
```

```
100)
        print(result)
        assert result.avg_time < 0.01


class TestMultiplayerPerformance:
    """Test performance of multiplayer game logic."""

    def test_update_multiplayer_match(self, pygame_env):
        factory = pygame_env["factory"]
        level = build_two_player_level(factory, (960, 640))
        match = TwoPlayerMatch(level)
        frame1 = InputFrame(movement=(1, 0))
        frame2 = InputFrame(movement=(-1, 0))

        def update():
            match.update(frame1, frame2, 0.016)

        result = measure_performance("Update Multiplayer Match",
update, 500)
        print(result)
        assert result.avg_time < 0.005

    def test_reset_multiplayer_match(self, pygame_env):
        factory = pygame_env["factory"]

        def reset():
            level = build_two_player_level(factory, (960, 640))
            TwoPlayerMatch(level)

        result = measure_performance("Reset Multiplayer Match", reset,
100)
        print(result)
        assert result.avg_time < 0.01


class TestFullGameLoopSimulation:
    """Test performance of a simulated game loop."""

    def test_simulate_game_frame_logic(self, pygame_env):
        factory = pygame_env["factory"]
        level = build_default_level(factory, (960, 640))
        player = level.player
        frame = InputFrame(movement=(1, 0))

        def simulate():
            player.update(frame, 0.016, level.bounds)
            for coin in level.coins:
                coin.update(0.016)
            for coin in level.coins[:]:
                if player.rect.colliderect(coin.rect):
                    level.coins.remove(coin)
            for hazard in level.hazards:
                player.rect.colliderect(hazard.rect)

        result = measure_performance("Simulate Game Frame (Logic)",
simulate, 200)
        print(result)
```

```
        assert result.avg_time < 0.005

    def test_simulate_game_frame_render(self, pygame_env):
        factory = pygame_env["factory"]
        drawing = pygame_env["drawing"]
        level = build_default_level(factory, (960, 640))

        def simulate():
            for coin in level.coins:
                coin.draw(drawing)
            for hazard in level.hazards:
                hazard.draw(drawing)
            level.player.draw(drawing)

        result = measure_performance("Simulate Game Frame (Render)",
simulate, 200)
        print(result)
        assert result.avg_time < 0.01
```

3. Результаты тестирования

Ниже представлен формат вывода результатов для одного из тестов.

```
Test: Update Player
================================================================================
Iterations:              1000
Total Time:             0.001 s
--------------------------------------------------------------------------------
Average Time:           0.001 ms
Median (P50):           0.001 ms
95th Percentile:        0.001 ms
99th Percentile:        0.002 ms
Min Time:               0.001 ms
Max Time:               0.046 ms
Std Deviation:          0.001 ms
--------------------------------------------------------------------------------
Throughput:            953016 ops/sec
================================================================================
PASSED          [ 28%]==========================================================
```

Рисунок 1. Формат вывода результатов теста

4. Анализ результатов

Сильные стороны:

1) Логика кадра выполняется за ~0.002 мс — это в ~8 000 раз быстрее целевого порога в 16.6 мс. Игра имеет огромный запас по производительности логики.

2) Паттерн Prototype в фабрике работает отлично: клонирование монет и препятствий через clone_at() быстрее, чем создание с нуля.

3) Коллизии через pygame.Rect.colliderect крайне эффективны — 10 проверок занимают 0.001 мс.

4) Обновление сущностей практически бесплатное: Coin.update достигает 7 млн ops/sec.

Зоны внимания:

1) Рендеринг — главное узкое место. Отрисовка полного кадра занимает ~0.1 мс при текущем количестве объектов, но при масштабировании (100+ объектов) это станет критично.

2) Draw Hazard медленнее других (0.013 мс против 0.003 мс для монеты) из-за множества примитивов: rectangle + 2×polygon + 4×circle + line.

3) P99 заметно выше P50 в тестах рендеринга (например, 0.578 мс против 0.300 мс в "Draw 10 of Each"), что указывает на возможные всплески из-за GC или переключений контекста.

4) SettingsManager — синглтон с I/O операциями; при частых переключениях настроек возможна деградация из-за записи JSON на диск.

Общий вердикт: Приложение полностью удовлетворяет требованиям производительности для 60 FPS. Текущий запас позволяет увеличить количество объектов на экране в 10–50 раз без потери плавности.

Вывод:

Проделав работу, применил технологию тестирования производительности для разработанного приложения.